
Learning long-range spatial dependencies with horizontal gated recurrent units

Drew Linsley, Junkyung Kim, Vijay Veerabadrán, Charlie Windolf, Thomas Serre

Carney Institute for Brain Science

Department of Cognitive Linguistic & Psychological Sciences

Brown University

Providence, RI 02912

{drew_linsley,junkyung_kim,vijay_veerabadrán,thomas_serre}@brown.edu

Abstract

Progress in deep learning has spawned great successes in many engineering applications. As a prime example, convolutional neural networks, a type of feedforward neural networks, are now approaching – and sometimes even surpassing – human accuracy on a variety of visual recognition tasks. Here, however, we show that these neural networks and their recent extensions struggle in recognition tasks where co-dependent visual features must be detected over long spatial ranges. We introduce a visual challenge, Pathfinder, and describe a novel recurrent neural network architecture called the horizontal gated recurrent unit (hGRU) to learn intrinsic horizontal connections – both within and across feature columns. We demonstrate that a single hGRU layer matches or outperforms all tested feedforward hierarchical baselines including state-of-the-art architectures with orders of magnitude more parameters.

1 Introduction

Consider Fig. 1a which shows a sample image from a representative segmentation dataset [1] (left) and the corresponding contour map produced by a state-of-the-art deep convolutional neural network (CNN) [2] (right). Although this task has long been considered challenging because of the need to integrate global contextual information with inherently ambiguous local edge information, modern CNNs are capable to detect contours in natural scenes at a level that rivals that of human observers [2–6]. Now, consider Fig. 1b which depicts a variant of a visual psychology task referred to as “Pathfinder” [7]. Reminiscent of the everyday task of reading a subway map to plan a commute (Fig. 1c), the goal in Pathfinder is to determine if two white circles in an image are connected by a path. These images are visually simple compared to natural images like the one shown in Fig. 1a, and the task is indeed easy for human observers [7]. Nonetheless, we will demonstrate that modern CNNs struggle to solve this task.

Why is it that a CNN can accurately detect contours in a natural scene like Fig. 1a but also struggle to integrate paths in the stimuli shown in Fig. 1b? In principle, the ability of CNNs to learn such long-range spatial dependencies is limited by their localized receptive fields (RFs) – hence the need to consider deeper networks because they allow the buildup of larger and more complex RFs. Here, we use a large-scale analysis of CNN performance on the Pathfinder challenge to demonstrate that simply increasing depth in feedforward networks constitutes an inefficient solution to learning the long-range spatial dependencies needed to solve the Pathfinder challenge.

An alternative solution to problems that stress long-range spatial dependencies is provided by biology. The visual cortex contains abundant horizontal connections which mediate non-linear interactions between neurons across distal regions of the visual field [8, 9]. These intrinsic connections, popularly

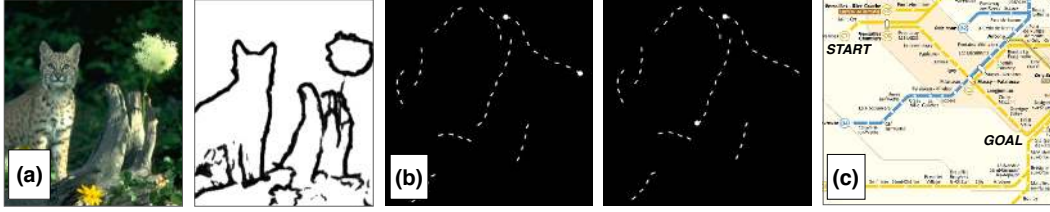


Figure 1: State-of-the-art CNNs excel at detecting contours in natural scenes, but they are strained by a task that requires the detection of long-range spatial dependencies. (a) Representative contour detection performance of a leading neural network architecture [23]. (b) Exemplars from the Pathfinder challenge: a task consisting of synthetic images which are parametrically controlled for long-range dependencies. (c) Long-range dependencies similar to those in the Pathfinder challenge are critical for everyday behaviors, such as reading a subway map to navigate a city.

called “association fields”, are thought to form the main substrate for mechanisms of contour grouping according to Gestalt principles, by mutually exciting colinear elements while also suppressing clutter elements that do not form extended contours [10–15]. Such “extra-classical receptive field” mechanisms, mediated by horizontal connections, allow receptive fields to adaptively “grow” without additional processing depth. Building on previous computational neuroscience work [e.g., 10, 16–19], our group has recently developed a recurrent network model of classical and extra-classical receptive fields that is constrained by the anatomy and physiology of the visual cortex [20]. The model was shown to account for diverse visual illusions providing computational evidence for a novel canonical circuit that is shared across visual modalities.

Here, we show how this computational neuroscience model can be turned into a modern end-to-end trainable neural network module. We describe an extension of the popular gated recurrent unit (GRU) [21], which we call the horizontal GRU (hGRU). Unlike CNNs, which exhibit a sharp decrease in accuracy for increasingly long paths, we show that the hGRU is highly effective at solving the Pathfinder challenge with just *one layer* and a fraction of the number of parameters and training samples needed by CNNs. We further find that, when trained on natural scenes, the hGRU learns connection patterns that coarsely resemble anatomical patterns of horizontal connectivity found in the visual cortex, and exhibits a detection profile that strongly correlates with human behavior on a classic contour detection task [22].

Related work Much previous work on recurrent neural networks (RNNs) has focused on modeling sequences with learnable gates in the form of long-short term memory (LSTM) units [24] or gated recurrent units (GRUs) [21]. RNNs have also been extended to learning spatial dependencies in static images with broad applications [25–29]. In this approach, images are transformed into one-dimensional sequences that are used to train an RNN. In recent years, several approaches have introduced convolutions into RNNs, using the recursive application of convolutional filters as a method for increasing the depth of processing through time on tasks like object recognition and super-resolution without additional parameters [30–32]. Other groups have constrained these convolutional-RNNs with insights from neuroscience and cognitive science, engineering specific patterns of connectivity between processing layers [33–36]. The proposed hGRU builds on this line of biologically-inspired implementations of RNNs, adding connectivity patterns and circuit mechanisms that are typically found in computational neuroscience models of neural circuits [e.g., 10, 16–20].

Another class of models related to our proposed approach is Conditional Random Fields (CRFs), probabilistic models aimed at explicitly capturing associations between nearby features. The connectivity implemented in CRFs is similar to the horizontal connections used in the hGRU, and has been successfully applied as a post-processing stage in visual tasks such as segmentation [37, 38] to smooth out and increase the spatial resolution of prediction maps. Recently, such probabilistic methods have been successfully incorporated in a generative vision model shown to break text-based CAPTCHAs [39]. Originally formulated as probabilistic models, CRFs can also be cast as RNNs [40].

2 Horizontal gated recurrent units (hGRUs)

Original contextual neural circuit model We begin by referencing the recurrent neural model of contextual interactions developed by Mély et al. [20]. Below we adapted the model notations to a computer vision audience. Model units are indexed by their 2D positions (x, y) and feature channel k . Neural activity is governed by the following differential equations (see Supp. Material for the full treatment):

$$\begin{aligned}\eta \dot{H}_{xyk}^{(1)} + \epsilon^2 H_{xyk}^{(1)} &= \left[\xi X_{xyk} - (\alpha H_{xyk}^{(1)} + \mu) C_{xyk}^{(1)} \right]_+ \\ \tau \dot{H}_{xyk}^{(2)} + \sigma^2 H_{xyk}^{(2)} &= \left[\gamma C_{xyk}^{(2)} \right]_+.\end{aligned}\tag{1}$$

where

$$\begin{aligned}C_{xyk}^{(1)} &= (\mathbf{W}^I * \mathbf{H}^{(2)})_{xyk} \\ C_{xyk}^{(2)} &= (\mathbf{W}^E * \mathbf{H}^{(1)})_{xyk},\end{aligned}$$

Here, $\mathbf{X} \in \mathbb{R}^{W \times H \times K}$ is the feedforward drive (i.e., neural responses to a stimulus), $\mathbf{H}^{(1)} \in \mathbb{R}^{W \times H \times K}$ is the recurrent circuit input, and $\mathbf{H}^{(2)} \in \mathbb{R}^{W \times H \times K}$ the recurrent circuit output. Modeling input and output states separately allows for the implementation of a particular form of inhibition known as “shunting” (or divisive) inhibition. Unlike the excitation in the model which acts linearly on a unit’s input, inhibition acts on a unit’s output and hence, regulates the unit response non-linearly (i.e., given a fixed amount of inhibition and excitation, inhibition will increase with the unit’s activity unlike excitation which will remain constant).

The convolutional kernels $\mathbf{W}^I, \mathbf{W}^E \in \mathbb{R}^{S \times S \times K \times K}$ describe inhibitory vs. excitatory hypercolumn connectivity (constrained by anatomical data¹). The scalar parameters μ and α control linear and quadratic (i.e., shunting) inhibition by $\mathbf{C}^{(1)} \in \mathbb{R}^{W \times H \times K}$, γ scales excitation by $\mathbf{C}^{(2)} \in \mathbb{R}^{W \times H \times K}$, and ξ scales the feedforward drive. Activity at each stage is linearly rectified (ReLU) $[\cdot]_+ = \max(\cdot, 0)$. Finally, η, ϵ, τ and σ are time constants. To make this model amenable to modern computer vision applications, we set out to develop a version where all parameters could be trained from data. If we let $\eta = \tau$ and $\sigma = \epsilon$ for symmetry and apply Euler’s method to Eq. 1 with a time step of $\Delta t = \eta/\epsilon^2$, then we obtain the discrete-time equations:

$$\begin{aligned}H_{xyk}^{(1)}[t] &= \epsilon^{-2} \left[\xi X_{xyk} - (\alpha H_{xyk}^{(1)}[t-1] + \mu) C_{xyk}^{(1)}[t] \right]_+ \\ H_{xyk}^{(2)}[t] &= \epsilon^{-2} \left[\gamma C_{xyk}^{(2)}[t] \right]_+.\end{aligned}\tag{2}$$

Here, $\cdot[t]$ denotes the approximation at the t -th discrete timestep. This results in a trainable convolutional recurrent neural network (RNN) which performs Euler integration of a dynamical system similar to the neural model of [20].

hGRU formulation We build on Eq. 2 to introduce the hGRU – a model with the ability to learn complex interactions between units via horizontal connections within a single processing layer (Fig. 2). The hGRU extends the derivation from Eq. 2 with three modifications that improve the training of the model with gradient descent and its expressiveness². (i) We introduce learnable gates, borrowed from the gated recurrent unit (GRU) framework (see Supp. Material for the full derivation from Eq. 2). (ii) The hGRU makes the operations for computing $\mathbf{H}^{(2)}$ (excitation) symmetric with those of $\mathbf{H}^{(1)}$ (inhibition), providing the circuit the ability to learn how to implement linear and quadratic interactions at each of these processing stages. (iii) To control unstable gradients, the hGRU uses a squashing pointwise non-linearity and a learned parameter to globally scale activity at every processing timestep (akin to a constrained version of the recurrent batchnorm [41]).

¹There are four separate connectivity patterns in [20] to describe inhibition vs. excitation and near vs. far interactions between units. We combine these into a separate inhibitory vs. excitatory kernels to simplify notation.

²These modifications involved relaxing several constraints from the original neuroscience model that are less useful for solving the tasks investigated here (see Supp. Material for performance of an hGRU with constrained inhibition and excitation.)

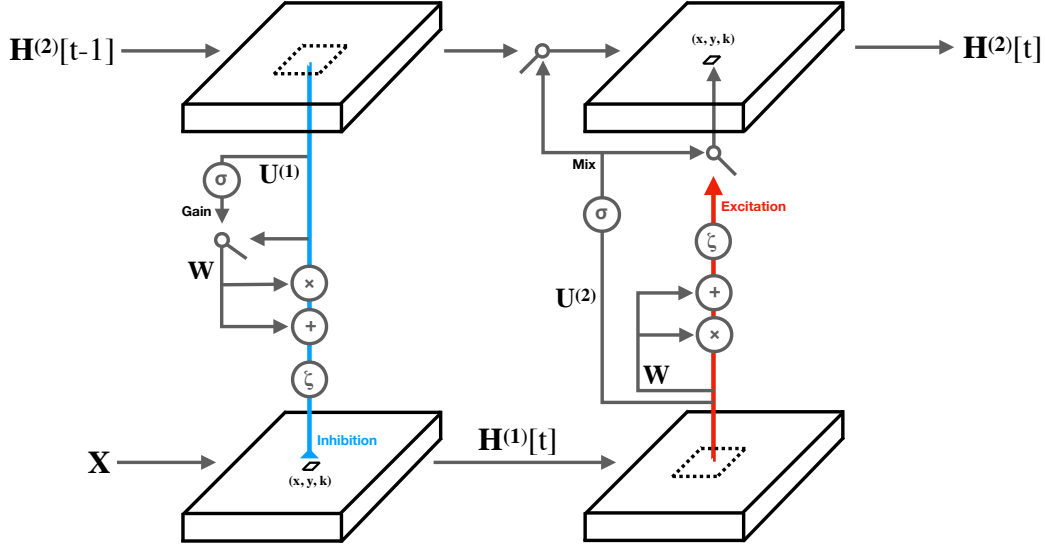


Figure 2: The hGRU circuit. The hGRU can learn highly non-linear interactions between spatially neighboring units in the feedforward drive $\mathbf{X}$, which are encoded in its hidden state $\mathbf{H}^{(2)}$. This computation involves two stages, which are inspired by a recurrent neural circuit of horizontal connections [20]. First, the horizontal inhibition (blue) is calculated by applying a gain to $\mathbf{H}^{(2)}[t-1]$, and convolving the resulting activity with the kernel $\mathbf{W}$ which characterizes these interactions. Linear (+ symbol) and quadratic ($\times$ symbol) operations control the convergence of this inhibition onto $\mathbf{X}$. Second, the horizontal excitation (red) is computed by convolving $\mathbf{H}^{(1)}[t]$ with $\mathbf{W}$. Another set of linear and quadratic operations modulate this activity, before it is mixed with the persistent hidden state $\mathbf{H}^{(2)}[t-1]$. Note that the excitation computation involves an additional “peephole” connection, not depicted here. Small solid-line squares within the hypothetical activities that the circuit operates on denote the unit indexed by 2D position (x, y) and feature channel k , whereas dotted-line squares depict the unit’s receptive field (a union of both classical and extra-classical definitions) in the previous activity.

In our hGRU implementation, the feedforward drive $\mathbf{X}$ corresponds to activity from a preceding convolutional layer. The hGRU encodes spatial dependencies between feedforward units via its (time-varying) hidden states $\mathbf{H}^{(1)}$ and $\mathbf{H}^{(2)}$. Updates to the hidden states are managed using two activities, referred to as the reset and update “gates”: $\mathbf{G}^{(1)}$ and $\mathbf{G}^{(2)}$. These activities are derived from convolutions, denoted by $*$, between the kernels $\mathbf{U}^{(1)}, \mathbf{U}^{(2)} \in \mathbb{R}^{I \times I \times K \times K}$ and hidden states $\mathbf{H}^{(1)}$ and $\mathbf{H}^{(2)}$, shifted by biases $\mathbf{b}^{(1)}, \mathbf{b}^{(2)} \in \mathbb{R}^{I \times I \times K}$, respectively. The pointwise non-linearity σ is applied to each activity, normalizing them in the range $[0, 1]$. Because these activities are real-valued, we hereafter refer to the reset gate as the “gain”, and the update gate as the “mix”.

Horizontal interactions between units are calculated by the kernel $\mathbf{W} \in \mathbb{R}^{S \times S \times K \times K}$, where S describes the spatial extent of these connections in a single timestep (Fig. 2; but see Supp. Material for a version with separate kernels for excitation vs. inhibition, as in Eq. 2). Consistent with computational models of neural circuits (e.g., [10, 16–20]), $\mathbf{W}$ is constrained to have symmetric weights between channels, such that the weight $W_{x_0+\Delta x, y_0+\Delta y, k_1, k_2}$ is equal to the weight $W_{x_0+\Delta x, y_0+\Delta y, k_2, k_1}$ where x_0 and y_0 denote the center of the kernel. This constraint reduces the number of learnable parameters by nearly half vs. a normal convolutional kernel. Hidden states $\mathbf{H}^{(1)}$ and $\mathbf{H}^{(2)}$ are recomputed via horizontal interactions at every timestep $t \in [0, T]$. We begin by describing computation of $\mathbf{H}^{(1)}[t]$:

$$\mathbf{G}^{(1)}[t] = \sigma(\mathbf{U}^{(1)} * \mathbf{H}^{(2)}[t-1] + \mathbf{b}^{(1)}) \quad (3)$$

$$C_{xyk}^{(1)}[t] = (\mathbf{W} * (\mathbf{G}^{(1)}[t] \odot \mathbf{H}^{(2)}[t-1]))_{xyk} \quad (4)$$

$$H_{xyk}^{(1)}[t] = \zeta(X_{xyk} - C_{xyk}^{(1)}[t](\alpha_k H_{xyk}^{(2)}[t-1] + \mu_k)) \quad (5)$$

Channels in $\mathbf{H}^{(2)}[t-1]$ are first modulated by the gain³ $\mathbf{G}^{(1)}[t]$. The resulting activity is convolved with $\mathbf{W}$ to compute $\mathbf{C}^{(1)}[t]$, which is the horizontal inhibition of the hGRU at this timestep. This inhibition is applied to $\mathbf{X}$ via the parameters $\boldsymbol{\mu}$ and $\boldsymbol{\alpha}$, which are k -dimensional vectors that respectively scale linear and quadratic (akin to shunting inhibition described in Eq. 1) terms of the horizontal interaction with $\mathbf{X}$. The pointwise ζ is a hyperbolic tangent that squashes activity into the range $[-1, 1]$ (but see Supp. Material for a hGRU with a rectified linearity). Importantly, in contrast to the original circuit, in this formulation the update to $\mathbf{H}^{(1)}[t]$ (Eq. 5) is calculated by combining horizontal connection contributions of $\mathbf{C}^{(1)}[t]$ with $\mathbf{H}^{(2)}[t-1]$ rather than $\mathbf{H}^{(1)}[t-1]$, which we found improved learning on the visual tasks explored here.

The updated $\mathbf{H}^{(1)}[t]$ is next used to calculate $\mathbf{H}^{(2)}[t]$.

$$G_{xyk}^{(2)}[t] = \sigma((\mathbf{U}^{(2)} * \mathbf{H}^{(1)}[t])_{xyk} + b_k^{(2)}) \quad (6)$$

$$C_{xyk}^{(2)}[t] = (\mathbf{W} * \mathbf{H}^{(1)}[t])_{xyk} \quad (7)$$

$$\tilde{H}_{xyk}^{(2)}[t] = \zeta(\kappa_k H_{xyk}^{(1)}[t] + \beta_k C_{xyk}^{(2)}[t] + \omega_k H_{xyk}^{(1)}[t] C_{xyk}^{(2)}[t]) \quad (8)$$

$$H_{xyk}^{(2)}[t] = \eta_t (H_{xyk}^{(2)}[t-1](1 - G_{xyk}^{(2)}[t]) + \tilde{H}_{xyk}^{(2)}[t] G_{xyk}^{(2)}[t]) \quad (9)$$

The mix $\mathbf{G}^{(2)}[t]$ is calculated by convolving $\mathbf{U}^{(2)}[t]$ with $\mathbf{H}^{(1)}[t]$, followed by the addition of $\mathbf{b}^{(2)}$. The activity $\mathbf{C}^{(2)}[t]$ represents the excitation of horizontal connections onto the newly-computed $\mathbf{H}^{(1)}[t]$. Linear and quadratic contributions of horizontal interactions at this stage are controlled by the k -dimensional parameters $\boldsymbol{\kappa}$, $\boldsymbol{\omega}$, and $\boldsymbol{\beta}$. The parameters $\boldsymbol{\kappa}$ and $\boldsymbol{\omega}$ control the linear and quadratic contributions of horizontal connections to $\tilde{\mathbf{H}}^{(2)}[t]$. The parameter $\boldsymbol{\beta}$ is a gain applied to $\mathbf{C}^{(2)}[t]$, giving $\mathbf{W}$ an additional degree of freedom in expressing this excitation. With this full suite of interactions, the hGRU can in principle implement both a linear and a quadratic form of excitation (i.e., to assess self-similarity), each of which play specific computational roles in perception [42]. Note that the inclusion of $\mathbf{H}^{(1)}[t]$ in Eq. 8 functions as a “peephole” connection between it and $\tilde{\mathbf{H}}^{(2)}[t]$. Finally, the mix $\mathbf{G}^{(2)}$ integrates the candidate $\tilde{\mathbf{H}}_t^{(2)}$ with $\mathbf{H}_t^{(2)}$. The learnable T -dimensional parameter $\boldsymbol{\eta}$, which we refer to as a time-gain, helps control unstable gradients during training. This time-gain modulates $\mathbf{H}_t^{(2)}$ with the scalar, η_t , which as we show in our experiments below improves model performance.

3 The Pathfinder challenge

We evaluated the limits of feedforward and recurrent architectures on the “Pathfinder challenge”, a synthetic visual task inspired by cognitive psychology [7]. The task, depicted in Fig. 1b, involves detecting whether two circles are connected by a path. This is made more difficult by allowing target paths to curve and introducing multiple shorter unconnected “distractor” paths. The Pathfinder challenge involves three separate datasets, for which the length of paths and distractors are parametrically increased. This challenge therefore screens models for their effectiveness in detecting complex long-range spatial relationships in cluttered scenes.

Stimulus design Pathfinder images were generated by placing oriented “paddles” on a canvas to form dashed paths. Each image contained two paths made of a fixed number of paddles and multiple distractors made of one third as many paddles. Positive examples were generated by placing two circles at the ends of a single path (Fig. 1b, left) and negative examples by placing one circle at the end of each of the paths (Fig. 1b, right). The paths were curved and variably shaped, with the possible number of shapes exponential to the path length. The Pathfinder challenge consisted of three datasets, in which path and distractor length was successively increased, and with them, the overall task difficulty. These datasets had path lengths of 6, 9 and 14 paddles, and each contained 1,000,000 unique images of 150×150 pixels. See Supp. Material for a detailed description of the stimulus generation procedure.

³GRU gate activities are often a function of a hidden state and $\mathbf{X}[t]$. Because the feedforward drive here is constant w.r.t. time, we omit it from these calculations. In practice, its inclusion did not affect performance.

Model implementation We performed a large-scale analysis of the effectiveness of feedforward and recurrent computations on the Pathfinder challenge. We controlled for the effects of idiosyncratic model specifications by using a standard architecture, consisting of “input”, “feature extraction”, and “readout” processing stages. Swapping different feedforward or recurrent layers into the feature extraction stage let us measure the relative effectiveness of each on the challenge. All models except for state-of-the-art “residual networks” (ResNets) [43] and per-pixel prediction architectures were embedded in this architecture, and these exceptions are detailed below. See Supp. Material for a detailed description of the input and readout stage. Models were trained on each Pathfinder challenge dataset (Fig. 3d), with 90% of the images used for training (900,000) and the remainder for testing (100,000). We measured model performance in two ways. First, as the accuracy on test images. Second, as the “area under the learning curve” (ALC), or mean accuracy on the test set evaluated after every 1000 batches of training, which summarized the rate at which a model learned the task. Accuracy and ALC were taken from the model that achieved the highest accuracy across 5 separate runs of model training. All models were trained for two epochs except for the ResNets, which were trained for four. Model training procedures are detailed in Supp. Material.

Recurrent models We tested 6 different recurrent layers in the feature extraction stage of the standard architecture: hGRUs with 8, 6, and 4-timesteps of processing; a GRU; and hGRUs with lesions applied to parameters controlling linear or quadratic horizontal interactions. Both the GRU and lesioned versions of the hGRU ran for 8 timesteps. These layers had 15×15 horizontal connection kernels (W) with an equal number of channels as their input layer (25 channels).

We observed 3 overarching trends: First, each model’s performance monotonically decreased, or “strained”, as path length increased. Increasing path length reduced model accuracy (Fig. 3a), and increased the number of batches it took to learn a task (Fig. 3b). Second, the 8-timestep hGRU was more effective than any other recurrent model, and it outperformed each of its lesioned variants as well as a standard GRU. Notably, this hGRU was strained the *least* by the Pathfinder challenge out of all tested models, with a negligible drop in accuracy as path length increased. This finding highlights the effectiveness of the hGRU for processing long-range spatial dependencies, and how the dynamics implemented by its linear and quadratic horizontal interactions are important. Third, hGRU performance monotonically decreased with processing time. This revealed a minimum number of timesteps that the hGRU needed to solve each Pathfinder dataset: 4 for the length-6 condition, 6 for the length-9 condition, and 8 for the length-14 condition (first vs. second columns in Fig. 3a). Such time-dependency in the Pathfinder task is consistent with the accuracy-reaction-time tradeoff found in humans as the distance between endpoints of a curve increases [7].

Feedforward models We screened an array of feedforward models on the Pathfinder challenge. Model performance revealed the importance of kernel size vs. kernel width, model depth, and feedforward operations for incorporating additional scene context for solving Pathfinder. Model construction began by embedding the feature extraction stage of the standard model with kernels of one of three different sizes: 10×10 , 15×15 , or 20×20 . These are referred to as small, medium, and large kernel models (Fig. 3). To control for the effect of network capacity on performance, the number of kernels given to each model was varied so that the number of parameters in each model configuration was equal to each other and the hGRU (36, 16, and 9 kernels). We also tested two other feedforward models that featured candidate operations for incorporating contextual information into local convolutional activities. One version used (2-pixel) dilated convolutions, which involves applying a stride to the kernel before convolving the input [44, 45], and has been found useful for many computer vision problems [38, 46, 47]. The other version applied a non-local operation to convolutional activities [48], which can introduce (non-recurrent) interactions between units in a layer. These operations were incorporated into the first feature extraction layer of the medium kernel (15×15 filter) model described above. We also considered deeper versions of each of the above “1-layer” models (referring to the depth of the feature extraction stage), stacking them to build 3- and 5-layer versions. This yielded a total of 15 different feedforward models.

Without exception, the performance of each feedforward model was significantly strained by the Pathfinder challenge. The magnitude of this straining was well predicted by model depth and size, and operations for incorporating additional contextual information made no discernible difference to the overall pattern of results. The 1-layer models were most effective on the 6-length Pathfinder dataset, but were unable to do better than chance on the remaining conditions. Increasing model capacity to 3 layers rescued the performance of all but the small kernel model on the 9-length Pathfinder dataset,

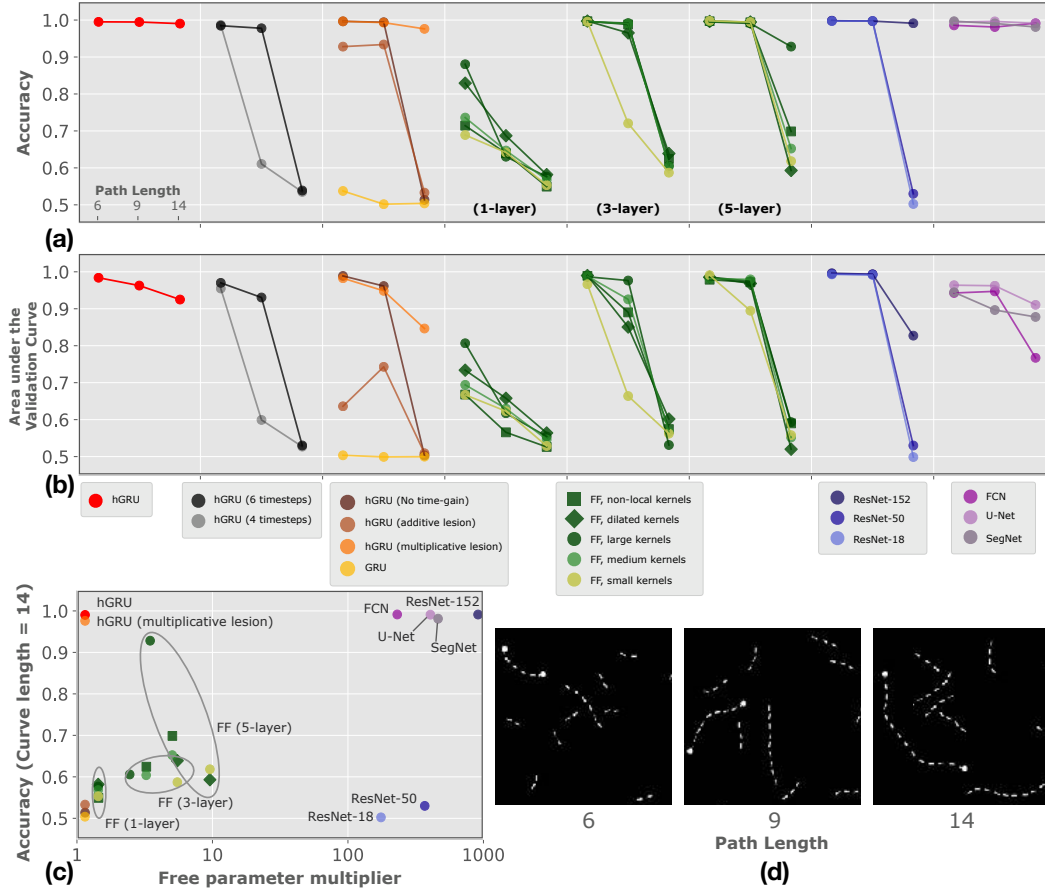


Figure 3: The hGRU efficiently learns long-range spatial dependencies that otherwise strain feedforward architectures. **(a)** Model accuracy is plotted for the three Pathfinder challenge datasets, which featured paths of 6- 9- and 14-paddles. Each panel depicts the accuracy of a different model class after training on for each pathfinder dataset (see Supp. Material for additional models). Only the hGRU and state-of-art models for classification (the two right-most panels) approached perfect accuracy on each dataset. **(b)** Measuring the area under the learning curve (ALC) of each model (mean accuracy) demonstrates that the rate of learning achieved by the hGRU across the Pathfinder challenge is only rivaled by the U-Net architecture (far right). **(c)** The hGRU is significantly more parameter-efficient than feedforward models at the Pathfinder challenge, with its closest competitors needing at least $200\times$ the number of parameters to match its performance. The x-axis shows the number of parameters in each model versus the hGRU, as a multiple of the latter. The y-axis depicts model accuracy on the 14-length Pathfinder dataset. **(d)** Pathfinder challenge exemplars of different path lengths (all are positive examples).

but even then did little to improve performance on the 14-length dataset. Of the 5-layer models, only the large kernel configuration came close to solving the 14-length dataset. The ALC of this model, however, demonstrates that its rate of learning was slow, especially compared to the hGRU (Fig. 3b). The failures of these feedforward models is all the more striking when considering that each had between $1\times$ and $10\times$ the number of parameters as the hGRU (Fig. 3c, compare the red and green markers).

Residual networks We reasoned that if the performance of feedforward models on the Pathfinder challenge is a function of model depth, then state-of-the-art networks for object recognition with many times the number of layers should easily solve the challenge. We tested this possibility by

training ResNets with 18, 50, and 152 layers on the Pathfinder challenge. Each model was trained “from scratch” with standard weight initialization [49], and given additional epochs of training (4) to learn the task because of their large number of parameters. However, even with this additional training, only the deepest 152-layer ResNet was able to solve the challenge (Fig. 3a). Even so, the 152-layer ResNet was less efficient at learning the 14-length dataset than the hGRU (Fig. 3b), and achieved its performance with nearly $1000\times$ as many parameters (Fig. 3c; see Supp. Material for additional ResNet experiments).

Per-pixel prediction models We considered the possibility that CNN architectures for per-pixel prediction tasks, such as contour detection and segmentation, might be better suited to the Pathfinder challenge than those designed for classification. We therefore tested three representative per-pixel prediction models: the fully-convolutional network (FCN), the skip-connection U-Net, and the unpooling SegNet. These models used an encoder/decoder style architecture, which was followed by the readout processing stage of the standard architecture described above to make them suitable for Pathfinder classification. Encoders were the VGG16 [50], and each model was trained from scratch with Xavier initialized weights. Like the ResNets, these models were given 4 epochs of training to accommodate their large number of parameters.

The fully-convolutional network (FCN) architecture is one of the first successful uses of CNNs for per-pixel prediction [3, 44, 51, 52]. Decoders in these models use “ 1×1 ” convolutions to combine upsampled activity maps from several layers of the encoder. We created an FCN model which applied this procedure to the last layer of each of the 5 VGG16-convolution blocks. These activity maps were upsampled by learnable kernels, which were initialized with weights for bilinear interpolation. In contrast to the feedforward models discussed above, the FCN successfully learned all conditions in the Pathfinder challenge (Fig. 3a, purple circle). It did so less efficiently than the hGRU, however, with a lower ALC score on the 14-length dataset and $200\times$ as many free parameters (Fig. 3b).

Another approach to per-pixel prediction uses “skip connections” to connect specific layers of a model’s encoder to its decoder. This approach was first described in [44] as a method for more effectively merging coarse-layer information into a model’s decoder, and later extended to the U-Net [53]. We implemented a version of the U-Net architecture that had a VGG16 encoder and a decoder. The decoder consisted of 5 randomly initialized and learned upsampling layers, which had additive connections to the final convolutional layer in each of the encoder’s VGG16 blocks. Using standard VGG16 nomenclature to define one of these connections, this meant that “conv 4_3” activity from the encoder was added to the second upsampled activity map in the decoder. The U-Net was on par with the hGRU and the FCN at solving the Pathfinder challenge. It was also nearly as efficient as the hGRU in doing so (Fig. 3b), but used over $350\times$ as many parameters as the hGRU (Fig. 3c; see Supp. Materials for additional U-Net experiments).

Unpooling models eliminate the need for feature map upsampling by routing decoded activities to the locations of the winning max-pooling units derived from the encoder. Unpooling is also a leading approach for a variety of dense per-pixel prediction tasks, including segmentation, which is exemplified by SegNet [54]. We tested a SegNet on the Pathfinder challenge. This model has a decoder that mirrors its encoder, with unpooling operations replacing its pooling. The SegNet achieved high accuracy on each of the Pathfinder datasets, but was less efficient at learning them than the hGRU, with worse ALC scores across the challenge (Fig. 3b). The SegNet also featured the second-most parameters of any model tested, which was $400\times$ more than the hGRU.

4 Explaining biological horizontal connections with the hGRU

Statistical image analysis studies have suggested that cortical patterns of horizontal connections, commonly referred to as “association fields”, may reflect the geometric regularities of oriented elements present in natural scenes [55]. Because the hGRU is designed to capture such spatial regularities, we investigated whether it learned patterns of horizontal connections that resemble these association fields. We visualized the horizontal kernels learned by the hGRU to solve tasks (Fig. S6). When trained on the the Pathfinder challenge, hGRU kernels resembled the dominant patterns of horizontal connectivity in visual cortex. Prominent among these patterns are (1) the antagonistic near-excitatory vs. far-inhibitory surround organization also found in the visual cortex [56]; (2) the association field, with collinear excitation and orthogonal inhibition [8, 9]; and (3) other higher-order surround computations [57]. We also visualized these patterns after training the hGRU to detect

contours in the naturalistic BSDS500 image dataset [1]. These horizontal kernels took on similar patterns of connectivity, but with far more definition and regularity, suggesting that the hGRU learns best from natural scene statistics.

How well does the hGRU explain human psychophysics data? We tested this by recreating the synthetic contour detection dataset used in [22]. This task had human participants detect a contour formed by co-linearly aligned paddles in an array of randomly oriented distractors. Multiple versions of the task were created by varying the distance between paddles in the contour (5 conditions). Contour detection accuracy of the hGRU was recorded on each of dataset for comparison with participants in [22], whose responses were digitally extracted with WebPlotDigitizer from [22] and averaged ($N=2$). Plotting hGRU accuracy against the reported “detection score” revealed that increasing inter-paddle distance caused similar performance straining for both (Fig. S6b).

5 Discussion

The present study demonstrates that long-range spatial dependencies generally strain CNNs, with only very deep and state-of-the-art networks overcoming the visual variability introduced by long paths in the Pathfinder challenge. Although feedforward networks are generally effective at learning and detecting relatively rigid objects shown in well-defined poses, these models tend towards a brute-force solution when tasked with the recognition of less constrained structures, such as a path connecting two distant locations. This study adds to a body of work highlighting examples of routine visual tasks where CNNs fall short of human performance [58–62].

We demonstrate a solution to the Pathfinder challenge inspired by neuroscience. The hGRU leverages computational principles of visual cortical circuits to learn complex spatial interactions between units. For the Pathfinder challenge, this translates into an ability to represent the elements forming an extended path while ignoring surrounding clutter. We find that the hGRU can reliably detect paths of any tested length or form using just a single layer. This contrasts sharply with the successful state-of-the-art feedforward alternatives, which used much deeper architectures and orders of magnitude more parameters to achieve similar success. The key mechanisms underlying the hGRU’s performance are well known in computational neuroscience [10, 16–20]. However, these mechanisms have been typically overlooked in computer vision (but see [39] for a successful vision model using horizontal connections and shown to break text-based CAPTCHAs).

We also found that hGRU performance on the Pathfinder challenge is a function of the *amount of time* it was given for processing. This finding suggests that it concurrently expands the facilitative influence of one end of a target curve to the other while suppressing the influence of distractors. The performance of the hGRU on the Pathfinder challenge captures the iterative nature of computations used by our visual system during similar tasks [63] – exhibiting a comparable tradeoff between performance and processing-time [7]. Visual cortex is replete with association fields that are thought to underlie perceptual grouping [11, 64]. Theoretical models suggest that patterns of horizontal connections reflect the statistics of natural scenes, and here too we find that horizontal kernels in the hGRU learned from natural scenes resemble cortical patterns of horizontal connectivity, including association fields and the paired near-excitatory / far-inhibitory surrounds that may be responsible for many contextual illusions [20, 56]. The horizontal connections learned by the hGRU reproduce another aspect of human behavior, in which the saliency of a straight contour decreases as the distance between its paddles increases. This sheds light on a possible relationship between horizontal connections and saliency computation.

In summary, this work diagnoses a computational deficiency of feedforward networks, and introduces a biologically-inspired solution that can be easily incorporated into existing deep learning architectures. The weights and patterns of behavior learned by the hGRU appear consistent with those associated with the visual cortex, demonstrating its potential for establishing novel connections between machine learning, cognitive science, and neuroscience.

Acknowledgments

This research was supported by NSF early career award [grant number IIS-1252951] and DARPA young faculty award [grant number YFA N66001-14-1-4037]. Additional support was provided by the Carney Institute for Brain Science and the Center for Computation and Visualization (CCV) at Brown University.

References

- [1] P. Arbelaez, M. Maire, C. Fowlkes, and J. Malik. Contour detection and hierarchical image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(5):898–916, May 2011.
- [2] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu. Deeply-Supervised nets. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, pages 562–570, February 2015.
- [3] S. Xie and Z. Tu. Holistically-Nested edge detection. *International Journal of Computer Vision*, 125(1):3–18, December 2017.
- [4] Y. Liu, M. M. Cheng, X. Hu, K. Wang, and X. Bai. Richer convolutional features for edge detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 5872–5881, July 2017.
- [5] K.-K. Maninis, J. Pont-Tuset, P. Arbelaez, and L. Van Gool. Convolutional oriented boundaries: From image segmentation to High-Level tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):819–833, April 2018.
- [6] Y. Wang, X. Zhao, and K. Huang. Deep crisp boundaries. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3892–3900, 2017.
- [7] R. Houtkamp and P. R. Roelfsema. Parallel and serial grouping of image elements in visual perception. *Journal of Experimental Psychology: Human Perception and Performance*, 36(6):1443–1459, December 2010.
- [8] D. D. Stettler, A. Das, J. Bennett, and C. D. Gilbert. Lateral connectivity and contextual interactions in macaque primary visual cortex. *Neuron*, 36(4):739–750, November 2002.
- [9] K. S. Rockland and J. S. Lund. Intrinsic laminar lattice connections in primate visual cortex. *Journal of Comparative Neurology*, 216(3):303–318, May 1983.
- [10] S. Grossberg and E. Mingolla. Neural dynamics of perceptual grouping: textures, boundaries, and emergent segmentations. *Perception & Psychophysics*, 38(2):141–171, August 1985.
- [11] D. J. Field, A. Hayes, and R. F. Hess. Contour integration by the human visual system: Evidence for a local “association field”. *Vision Research*, 33(2):173–193, 1993.
- [12] G. W. Leshner and E. Mingolla. The role of edges and line-ends in illusory contour formation. *Vision Research*, 33(16):2253–2270, November 1993.
- [13] W. Li, V. Piëch, and C. D. Gilbert. Contour saliency in primary visual cortex. *Neuron*, 50(6):951–962, June 2006.
- [14] W. Li, V. Piëch, and C. D. Gilbert. Learning to link visual contours. *Neuron*, 57(3):442–451, February 2008.
- [15] S. W. Zucker. Stereo, shading, and surfaces: Curvature constraints couple neural computations. *Proceedings of the IEEE*, 102(5):812–829, May 2014.
- [16] P. Series, J. Lorenceau, and Y. Frégnac. The “silent” surround of V1 receptive fields: theory and experiments. *Journal of Physiology-Paris*, 97:453–474, 2003.
- [17] L. Zhaoping. Neural circuit models for computations in early visual cortex. *Current Opinion in Neurobiology*, 21(5):808–815, October 2011.
- [18] S. Shushruth, P. Mangapathy, J. M. Ichida, P. C. Bressloff, L. Schwabe, and A. Angelucci. Strong recurrent networks compute the orientation tuning of surround modulation in the primate primary visual cortex. *Journal of Neuroscience*, 32(1):308–321, 2012.
- [19] D. B. Rubin, S. D. Van Hooser, and K. D. Miller. The stabilized supralinear network: a unifying circuit motif underlying multi-input integration in sensory cortex. *Neuron*, 85(2):402–417, January 2015.

- [20] D. A. Mély, D. Linsley, and T. Serre. Complementary surrounds explain diverse contextual phenomena across visual modalities. *Psychological Review*, 125(5):769–784, October 2018.
- [21] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN Encoder–Decoder for statistical machine translation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734, Stroudsburg, PA, USA, 2014.
- [22] W. Li and C. D. Gilbert. Global contour saliency and local colinear interactions. *Journal of Neurophysiology*, 88(5):2846–2856, November 2002.
- [23] S. Xie and Z. Tu. Holistically-Nested edge detection. *International Journal of Computer Vision*, 125(1-3):3–18, December 2017.
- [24] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, November 1997.
- [25] A. Graves, S. Fernández, and J. Schmidhuber. Multi-dimensional recurrent neural networks. In *Proceedings of the International Conference on Artificial Neural Networks*, pages 549–558, Berlin, Heidelberg, 2007.
- [26] A. Graves and J. Schmidhuber. Offline handwriting recognition with multidimensional recurrent neural networks. In *Advances in Neural Information Processing Systems*, pages 545–552, 2009.
- [27] S. Bell, C. Lawrence Zitnick, K. Bala, and R. Girshick. Inside-outside net: Detecting objects in context with skip pooling and recurrent neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2874–2883, 2016.
- [28] L. Theis and M. Bethge. Generative image modeling using spatial LSTMs. In *Advances in Neural Information Processing Systems*, pages 1927–1935, 2015.
- [29] A. Van Den Oord, N. Kalchbrenner, and K. Kavukcuoglu. Pixel recurrent neural networks. In *Proceedings of the International Conference on Machine Learning*, vol. 48, pages 1747–1756, New York, NY, USA, 2016.
- [30] M. Liang and X. Hu. Recurrent convolutional neural network for object recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3367–3375, 2015.
- [31] Q. Liao and T. Poggio. Bridging the gaps between residual learning, recurrent neural networks and visual cortex. *arXiv preprint arXiv:1604.03640*, April 2016.
- [32] J. Kim, J. K. Lee, and K. M. Lee. Deeply-Recursive convolutional network for image Super-Resolution. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1637–1645, 2016.
- [33] C. J. Spoerer, P. McClure, and N. Kriegeskorte. Recurrent convolutional neural networks: A better model of biological object recognition. *Frontiers in Psychology*, 8:1551, September 2017.
- [34] W. Lotter, G. Kreiman, and D. Cox. Deep predictive coding networks for video prediction and unsupervised learning. In *Proceedings of the International Conference on Learning Representations*, 2017.
- [35] A. R. Zamir, T.-L. Wu, L. Sun, W. Shen, B. E. Shi, J. Malik, and S. Savarese. Feedback Networks. *arXiv preprint arxiv: 1612.09508*, December 2016.
- [36] A. Nayebi, D. Bear, J. Kubilius, K. Kar, S. Ganguli, D. Sussillo, J. J. DiCarlo, and D. L. K. Yamins. Task-Driven convolutional recurrent models of the visual system. *arXiv preprint arXiv:1807.00053*, June 2018.
- [37] I. Kokkinos. Pushing the boundaries of boundary detection using deep learning. In *Proceedings of the International Conference on Learning Representations*, 2016.

- [38] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):834–848, April 2018.
- [39] D. George, W. Lehrach, K. Kinsky, M. Lázaro-Gredilla, C. Laan, B. Marthi, X. Lou, Z. Meng, Y. Liu, H. Wang, A. Lavin, and D. S. Phoenix. A generative vision model that trains with high data efficiency and breaks text-based CAPTCHAs. *Science*, 358(6368), December 2017.
- [40] S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. S. Torr. Conditional random fields as recurrent neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1529–1537, 2015.
- [41] T. Cooijmans, N. Ballas, C. Laurent, Ç. Gülçehre, and A. Courville. Recurrent batch normalization. In *Proceedings of the International Conference on Learning Representations*, 2017.
- [42] D. R. Martin, C. C. Fowlkes, and J. Malik. Learning to detect natural image boundaries using local brightness, color, and texture cues. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(5):530–549, May 2004.
- [43] K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. In *Proceedings of the European Conference on Computer Vision, Amsterdam, The Netherlands*, pages 630–645, 2016.
- [44] J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3431–3440, 2015.
- [45] F. Yu and V. Koltun. Multi-Scale context aggregation by dilated convolutions. In *Proceedings of the International Conference on Learning Representations*, 2016.
- [46] T. Wang, M. Sun, and K. Hu. Dilated deep residual network for image denoising. In *29th IEEE International Conference on Tools with Artificial Intelligence, Boston, MA, USA*, pages 1272–1279, 2017.
- [47] R. Hamaguchi, A. Fujita, K. Nemoto, T. Imaizumi, and S. Hikosaka. Effective use of dilated convolutions for segmenting small object instances in remote sensing imagery. *CoRR*, abs/1709.00179, 2017.
- [48] X. Wang, R. Girshick, A. Gupta, and K. He. Non-Local neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.
- [49] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 1026–1034, Washington, DC, USA, 2015.
- [50] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *CoRR*, abs/1409.1556, 2014.
- [51] G. Papandreou, I. Kokkinos, and P.-A. Savalle. Modeling local and global deformations in deep learning: Epitomic convolution, multiple instance learning, and sliding window detection. In *Proceedings of the Conference on Computer Vision and Pattern Recognition*, pages 390–399, June 2015.
- [52] K.-K. Maninis, J. Pont-Tuset, P. Arbelaez, and L. Van Gool. Convolutional oriented boundaries: From image segmentation to High-Level tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, May 2017.
- [53] O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pages 234–241. Springer International Publishing, 2015.
- [54] V. Badrinarayanan, A. Kendall, and R. Cipolla. SegNet: A deep convolutional Encoder-Decoder architecture for image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(12):2481–2495, December 2017.

- [55] O. Ben-Shahar and S. Zucker. Geometrical computations explain projection patterns of long-range horizontal connections in visual cortex. *Neural Computation*, 16(3):445–476, March 2004.
- [56] S. Shushruth, L. Nurminen, M. Bijanzadeh, J. M. Ichida, S. Vanni, and A. Angelucci. Different orientation tuning of near- and far-surround suppression in macaque primary visual cortex mirrors their tuning in human perception. *Journal of Neuroscience*, 33(1):106–119, January 2013.
- [57] H. Tanaka and I. Ohzawa. Surround suppression of V1 neurons mediates orientation-based representation of high-order visual features. *Journal of Neurophysiology*, 101(3):1444–1462, March 2009.
- [58] J. Kim, M. Ricci, and T. Serre. Not-So-CLEVR: learning same-different relations strains feedforward neural networks. *Interface Focus*, 8(4):20180011, August 2018.
- [59] C. Szegedy et al. Intriguing properties of neural networks. In *Proceedings of the International Conference on Learning Representations*, 2014.
- [60] A. Nguyen, J. Yosinski, and J. Clune. Deep neural networks are easily fooled: High confidence predictions for unrecognizable images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 427–436, 2015.
- [61] A. Volokitin, G. Roig, and T. A. Poggio. Do deep neural networks suffer from crowding? In *Advances in Neural Information Processing Systems 30*, pages 5628–5638, 2017.
- [62] K. Ellis, A. Solar-Lezama, and J. Tenenbaum. Unsupervised learning by program synthesis. In *Advances in Neural Information Processing Systems 28*, pages 973–981, 2015.
- [63] P. R. Roelfsema and R. Houtkamp. Incremental grouping of image elements in vision. *Attention, Perception & Psychophysics*, 73(8):2542–2572, November 2011.
- [64] C. D. Gilbert and W. Li. Top-down influences on visual processing. *Nature Reviews Neuroscience*, 14(5):350–363, May 2013.
- [65] C. Tallec and Y. Ollivier. Can recurrent neural networks warp time? *arXiv preprint arxiv: 1804.11188*, March 2018.
- [66] J. W. Peirce. PsychoPy—Psychophysics software in python. *Journal of Neuroscience Methods*, 162(1):8–13, May 2007.
- [67] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*, pages 249–256, Sardinia, Italy, 2010.
- [68] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arxiv: 1412.6980*, December 2014.
- [69] M. Abadi et al. TensorFlow: A system for large-scale machine learning. In *Proceedings of the USENIX Conference on Operating Systems Design and Implementation*, pages 265–283, Berkeley, CA, USA, 2016.
- [70] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arxiv: 1502.03167*, February 2015.
- [71] R. K. Srivastava, K. Greff, and J. Schmidhuber. Highway networks. *arXiv preprint arxiv: 1505.00387*, May 2015.

Supplementary Material

Deriving the hGRU from the contextual neural circuit model

From Eq. 1, we obtain the following by rearranging the decay term:

$$\begin{aligned}\dot{H}_{xyk}^{(1)} &= -\eta^{-1}\epsilon^2 H_{xyk}^{(1)} + \eta^{-1} \left[\xi X_{xyk} - (\alpha H_{xyk}^{(1)} + \mu) C_{xyk}^{(1)} \right]_+ \\ \dot{H}_{xyk}^{(2)} &= -\eta^{-1}\sigma^2 H_{xyk}^{(2)} + \tau^{-1} \left[\gamma C_{xyk}^{(2)} \right]_+, \end{aligned} \quad (10)$$

where $[\cdot]_+ = \max(\cdot, 0)$ is the ReLU function.

Now, we would like to discretize this equation using Euler's method. We first simplify the above equation by choosing $\eta = \tau$ and $\sigma = \epsilon$:

$$\begin{aligned}\dot{H}_{xyk}^{(1)} &= -\eta^{-1}\epsilon^2 H_{xyk}^{(1)} + \eta^{-1} \left[\xi X_{xyk} - (\alpha H_{xyk}^{(1)} + \mu) C_{xyk}^{(1)} \right]_+ \\ \dot{H}_{xyk}^{(2)} &= -\eta^{-1}\epsilon^2 H_{xyk}^{(2)} + \eta^{-1} \left[\gamma C_{xyk}^{(2)} \right]_+, \end{aligned}$$

Now, apply Euler's method with timestep h . This gives the following difference equation:

$$\begin{aligned}H_{xyk}^{(1)}[t] &= H_{xyk}^{(1)}[t-1] + h \left(-\eta^{-1}\epsilon^2 H_{xyk}^{(1)}[t-1] + \eta^{-1} \left[\xi X_{xyk}[t-1] - (\alpha H_{xyk}^{(1)}[t-1] + \mu) C_{xyk}^{(1)}[t] \right]_+ \right) \\ H_{xyk}^{(2)}[t] &= H_{xyk}^{(2)}[t-1] + h \left(-\eta^{-1}\epsilon^2 H_{xyk}^{(2)}[t-1] + \eta^{-1} \left[\gamma C_{xyk}^{(2)}[t] \right]_+ \right). \end{aligned}$$

Distributing h , we get

$$\begin{aligned}H_{xyk}^{(1)}[t] &= H_{xyk}^{(1)}[t-1] - h \frac{\epsilon^2}{\eta} H_{xyk}^{(1)}[t-1] + h \eta^{-1} \left[\xi X_{xyk}[t-1] - (\alpha H_{xyk}^{(1)}[t-1] + \mu) C_{xyk}^{(1)}[t] \right]_+ \\ H_{xyk}^{(2)}[t] &= H_{xyk}^{(2)}[t-1] - h \frac{\epsilon^2}{\eta} H_{xyk}^{(2)}[t-1] + h \eta^{-1} \left[\gamma C_{xyk}^{(2)}[t] \right]_+. \end{aligned}$$

Now, notice that if we choose $h = \frac{\eta}{\epsilon^2}$, the first two terms on the RHS of each line will cancel:

$$\begin{aligned}H_{xyk}^{(1)}[t] &= \epsilon^{-2} \left[\xi X_{xyk}[t-1] - (\alpha H_{xyk}^{(1)}[t-1] + \mu) C_{xyk}^{(1)}[t] \right]_+ \\ H_{xyk}^{(2)}[t] &= \epsilon^{-2} \left[\gamma C_{xyk}^{(2)}[t] \right]_+. \end{aligned} \quad (11)$$

We now have a discrete-time approximation of the initial dynamical system, shown in Eq. 2. Note that this model can also be thought of as a convolutional RNN with ReLU nonlinearity. Because RNNs are difficult to train in practice, the state of the art is to incorporate learned "gates" to manage the flow of information over time [65].

$$\begin{aligned}G_{xyk}^{(1)}[t] &= \sigma \left(X_{xyk} + (\mathbf{U}^{(1)} * \mathbf{H}^{(2)}[n-1])_{xyk} + b_k^{(1)} \right) \\ G_{xyk}^{(1)}[t] &= \sigma \left(X_{xyk} + (\mathbf{U}^{(2)} * \mathbf{H}^{(1)}[n])_{xyk} + b_k^{(2)} \right), \end{aligned} \quad (12)$$

where σ is a squashing pointwise nonlinearity, $\mathbf{U}^{(\cdot)}$ are convolution kernels, and $b^{(\cdot)}$ are bias vectors. Applied to Eq. 11, these gates integrate past information with new computations more flexibly than the Euler integration above:

$$\begin{aligned}H_{xyk}^{(1)}[t] &= \epsilon^{-2} G_{xyk}^{(1)}[t] \left[\xi X_{xyk}[t-1] - (\alpha H_{xyk}^{(1)}[t-1] + \mu) C_{xyk}^{(1)}[t] \right]_+ \\ &\quad + (1 - G_{xyk}^{(1)}[t]) H_{xyk}^{(1)}[t-1] \\ H_{xyk}^{(2)}[t] &= \epsilon^{-2} G_{xyk}^{(2)}[t] \left[\gamma C_{xyk}^{(2)}[t] \right]_+ + (1 - G_{xyk}^{(2)}[t]) H_{xyk}^{(2)}[t-1] \end{aligned} \quad (13)$$

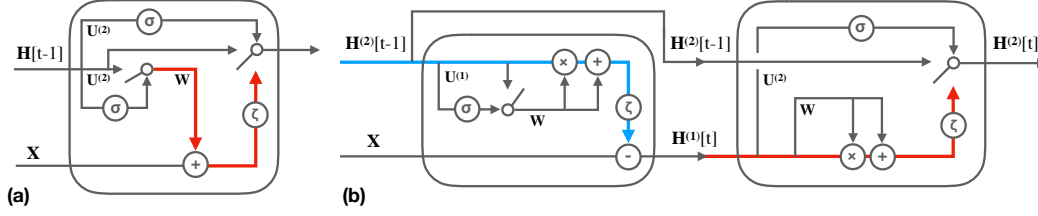


Figure S1: Circuit diagrams of the gated recurrent unit (GRU) and the hGRU showing the excitatory (red) and inhibitory (blue) flow of information. (a) A reference diagram of the GRU, demonstrating how the kernels $U^{(1)}$ and $U^{(2)}$ act as the “gain” and “mix” on the circuit’s computations: selecting feature channels from $H[t-1]$ to process on the current timestep, and determining how to mix a candidate $\tilde{H}[t]$ with $H[t-1]$ to produce $H[t]$. Candidate activities are calculated by applying a hyperbolic tangent ($\tanh$) to a linear combination of the feedforward drive X and the convolution of the kernel W with the new “gain”-modulated hidden state. (b) The same gain and mix are used in the hGRU in a slightly different configuration. Note that in the hGRU, the mix is a function of $\mathbf{H}^{(1)}[t]$ instead of $\mathbf{H}^{(2)}[t-1]$. The separate processing stages of the hGRU, inspired by the recurrent neural circuit of [20], are critical for learning the Pathfinder challenge.

This defines the simplest gated version of the RNN described in Eq. 11. As discussed in the main text, we modified the model in several other ways to improve its effectiveness on the visual tasks examined here: (1) We found that using the GRU-style “gain” (eq 3.) to control the first stage of horizontal interactions was more effective than using “mixing” gates in both stages. (2) We found it effective to replace the ϵ^{-2} scaling by the learned per-timestep parameter η , which can be thought of as a restricted application of batch normalization to the RNN’s hidden activity to control saturating activities stabilize training [41]. (3) We included both linear and quadratic forms of excitation in $\mathbf{H}^{(2)}$ for symmetry with $\mathbf{H}^{(1)}$. Enabling the model to spread excitation via both linear and quadratic propagation is potentially useful for propagating activity based on either first- (e.g., contrast strength) or second-order (e.g., self-similarity) statistical regularities. The linear and quadratic terms are scaled per-feature by the learnable parameters α and μ when calculating $\mathbf{H}^{(1)}$, and κ and ω , and ω in $\mathbf{H}^{(2)}$. See S1 for a comparison of circuit diagrams for the GRU vs. the hGRU.

The Pathfinder challenge

Overview The main goal of the Pathfinder challenge is to assess the ability of a computer vision system to determine whether two distant locations in an image, marked by white circles, are connected by a path. The stimulus dataset consists of binary images, each containing two white circles, two white ‘target’ paths, and multiple distractor paths, placed on a black background. All images used in our experiments are rendered on a 150×150 canvas. The task is inspired by [63].

Each image is generated by first sampling two target paths and then rendering multiple distractor paths. Each path consists of k co-circularly arranged identically shaped “paddles” of length l and thickness d (Fig. S2b). A paddle is a white oriented bar, characterized by the position of its center and its orientation, θ . Each path is generated inductively, namely, by first sampling the position of its “seed paddle” which serves as the end of the path and then iteratively adding new “trailing paddles” next to the seed paddle or the last trailing paddle. The high-level overview of the image generation algorithm is depicted in Fig. S2 and the list of image parameters in Table S2.

Positioning target paths The first stage of generating target paths involves randomly sampling the position of an invisible circle of radius r (Fig. S2a, step 1) within an empty image. Then, a randomly oriented line pivoted at the center of the circle makes two intersections with the circle. These intersections serve as the positions of two target paths’ seed paddles (Fig. S2a, step 2). This constraint ensures that target paths are always located within a sufficient proximity to each other. Without this constraint, target paths can be separated by an arbitrary distance, which may allow a model to make classification decision solely based on the distance between the white markers, for the markers will tend to be located much farther apart in negative examples. Then, two randomly

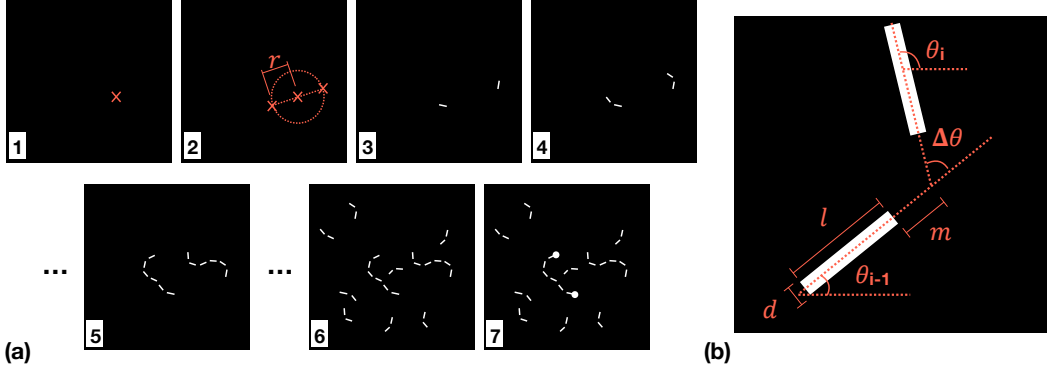


Figure S2: Image generator algorithm. (a) Depiction of image generation shown in seven steps. (b) Depiction of the geometric constraints for trailing paddle placement during path generation.

oriented paddles are rendered at each of the intersections (Fig. S2a, step 3), serving as seed paddles of the target paths.

Growing a path Trailing paddles are sequentially added to the end of each target path (Fig. S2a, step 4 and 5). Each trailing paddle is added by first sampling its orientation, θ_i , according to the probability distribution defined by the angle formed between the new and the previous trailing paddle, $\Delta\theta = \theta_i - \theta_{i-1}$:

$$P(\theta_i) = \frac{1}{Z} \max(\cos(c\Delta\theta), 0) \quad (14)$$

$$\text{where } Z = \int_{-\frac{\pi}{2}}^{\frac{\pi}{2}} \cos(\theta) d\theta = 2 \quad (15)$$

Note that the continuity parameter c determines the overall rigidity of a path by constraining the possible range of orientations of each new trailing paddle. With the sampled orientation, the new trailing paddle's position is determined such that the line extending from the trailing paddle intersects with the line extending from the new paddle m pixels away from the end of the two paddles while also forming an angle $\Delta\theta$ (Fig. S2b). Note that the parameter m determines the margin between adjacent paddles in a path. Because we do not allow paddles to cross or touch, we add new paddles to the two target paths in alternation to ensure that the shapes of one path does not restrict the shape of the other. The total number of target path paddles, or simply length of target paths, is denoted by the parameter k .

Additional distractor paths consisting of $\frac{k}{3}$ paddles are added to the image. Their generation process is identical to that of target paths, except that a seed paddle's location is independently sampled. The total number of distractor paths is chosen so that the total number of paddles in an image is 150. The usage of two target paths as well as distractor paths prevents any 'short-cut' solution to this problem, such as detecting a loose end of a path or a lone white circle as a local diagnostic cue for classification. Additionally, this design allows us to ensure that positive and negative images are indistinguishable in terms of the way paddles are arranged. This forces a model to perform classification solely based on the connectedness of the white circles.

As mentioned, the generator does not allow paddles to be too close to each other or make any contact. If a newly sampled paddle is making contact with or not separated from other paddles by at least m pixels, the generator rejects it and re-samples a paddle. In practice, this is done by applying circular dilation on the paddles using a kernel of radius m and checking if the dilated images of paddles has any overlap.

Shape variability Because adding an additional paddle to a path multiplicatively increases the number of possible shapes of the path, path shape variability is exponential to the total number of paddles in a path, k . Similarly, the continuity parameter c controls path shape variability in

Table S1: Image parameters in the Pathfinder challenge.

Notation	Definition
r	Radius of a circle
k	Target path length, or the number of paddles in a target path
l	Paddle length, in pixels
d	Paddle thickness, in pixels
m	Inter-paddle margin, in pixels
c	Continuity

an exponential manner because it scales the range of possible angles formed between every pair of adjacent paddles in a path. In our experiment, we vary path length k to examine how model performance changes as the shape variability of a path in a dataset increases. All images in the Pathfinder challenge are generated using Python and OpenCV.

The Synthetic Contours Dataset

In this section, we shall explain the motivation and construction procedure for the Synthetic Contours Dataset. We created the Synthetic Contours Dataset (SCD) to compare an hGRU pre-trained on the BSDS500 [1] for contour detection against human behavioral data as described in [22].

Dataset generation We generated the SCD by following closely, and extending the dataset construction procedure described in [22]. SCD enables us to parametrically control the position, orientation, and relative spacing of generated contours, resulting in 28 million images with practically infinite variability along these directions. All images in the SCD have a resolution of 256×256 pixels, spanning a radius of 4 visual degrees. Each contour image is rendered by placing “paddles” of length 0.1 visual degrees within uniformly spaced cells on a master grid. This master grid covers the entire image with a height and a width of 8 visual degrees. Each cell in the grid spans a height and width of 0.25 visual degrees. A total of 32 paddles are placed along every row/column of the master grid.

A contour of length l is generated in the master grid by filling l neighboring grid cells on any grid-diagonal, and each paddle connects the diagonally opposite points in their respective cells. This aligns the paddles in a collinear manner to form a contour of length l . In order to ensure that the majority of the contour stays within the image, its center is positioned within a maximum distance from the center of the image. This distance from the image center, known as eccentricity, is denoted as e . Once the contour is generated, all remaining unoccupied cells are filled with paddles each with an orientation $\theta \in [0, 2\pi]$ sampled from a uniform distribution.

Parametric dataset variability Contours in SCD are formed by regulating the dataset variability in 2 different directions: (1) contour length and (2) relative spacing. We denote the number of paddles present in a contour as its length. Each image in the SCD contains a contour with a length of either 5, 9, 14 or 17 paddles. The distance between neighboring paddles that form a contour is denoted as the relative spacing. While we vary the relative spacing, the global spacing between distractor paddles is maintained constant. Following [22], we apply a shear operation to the generated image by a shear factor SF , which controls how close or far the neighboring contour paddles are positioned. We generated images with 5 different relative-spacing conditions by varying SF between -0.6 and +0.6. A comparison between human performance and model performance on the task of contour detection can be found in Fig. S6. Example figures with different combinations of contour length and relative spacing are demonstrated in Fig. S3. All the images in SCD were generated using the Python Psychophysics toolbox, Psychopy [66].

Model architecture details

All of the recurrent models and feedforward models (shown in shades of green in Fig. 3a in the main text) tested in our study share the ‘input stage’ as the preprocessing module and the ‘readout stage’ as the classification module, which allows us to more directly compare the effectiveness of different architectures (called ‘feature extraction’ stage) at solving the Pathfinder challenge. Unless otherwise specified, all feedforward model weights were Xavier initialized [67]. Recurrent model kernels and

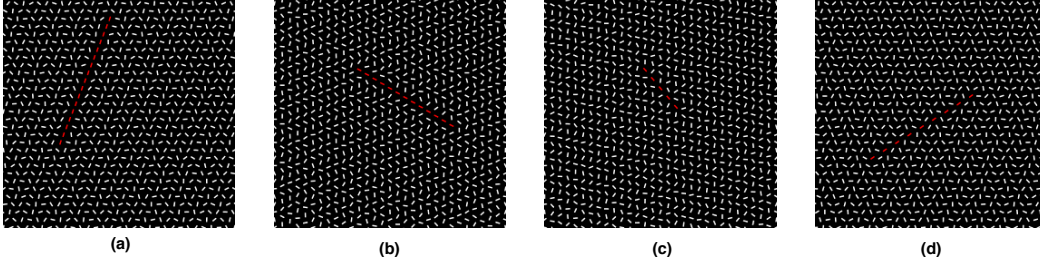


Figure S3: Examples of images from the SCD. Unlike in the dataset, contours in the figure are highlighted with a different color for the reader’s convenience. (a) Contour with $SF=-0.6$, $l=17$, closest relative spacing. (b) Contour with $SF=-0.4$, $l=14$. (c) Contour with $SF=0.2$, $l=9$. (d) Contour with $SF=0.6$, $l=9$, farthest relative spacing.

parameters were also initialized with Xavier, while their biases were chronos-initialized [65]. Hidden states of the recurrent models were also randomly initialized. Implementations of the hGRU can be found at https://github.com/serre-lab/hgru_share, and datasets are available upon request.

Standard cross entropy was used to compute loss. Models were trained with gradient descent using the Adaptive Moment Estimation (Adam) optimizer [68] with base learning rate of 10^{-3} on batches of 32 images. Our experiments are conducted using TensorFlow [69].

Input stage The input stage consists of a convolutional layer with 25 kernels of size 7×7 . The kernels are initialized as Gabor filters at 12 orientations and 2 phases, plus a radially symmetric difference-of-Gaussian filter. Filter activity undergoes a pointwise squaring non-linearity before being passed to the model-specific feature extraction stage. Note that none of the off-the-shelf models (residual networks and per-pixel prediction models) use this input stage.

Readout stage The readout stage takes the output from the final layer of the feature extraction stage and computes a two-dimensional likelihood vector for deriving a decision on an image from the Pathfinder challenge. The readout stage contained three parts: (1) a 1×1 convolutional filter that transforms the multi-channel output map from the feature extraction stage to a two-channel map, each corresponding to the positive/negative class. (2) A batch-normalized [70] global max pool, whose output represents in each channel the maximum activation value across space. (3) A linear classifier which maps the two-dimensional feature vector into a class decision. The configuration of this readout stage allows us to compare the accuracy between models designed for per-image prediction with those designed for per-pixel prediction. Note that residual networks do not use a readout stage as they are proposed as standalone image classification architectures.

Additional recurrent models We tested three other kinds of recurrent models that were not discussed in the main text. These were the hGRU with batchnorm (hGRU-BN), the hGRU with a nonnegative pointwise nonlinearity (hGRU-ReLU), and a two-layer GRU (GRU-2L).

The hGRU formulation in the main text incorporates a highly constrained form of the normalization discussed in [41], in which a learned scalar modulates activity at every timestep to help control unstable gradients. We found that incorporating the full batchnorm formulation of [41] in the hGRU also performed well on the Pathfinder challenge. We implemented the resulting hGRU-BN by adding batchnorm to multiple computations in the circuit. While it is possible to share batchnorm parameters across timesteps, the model performs better when they are not shared. In this formulation, $H^{(1)}[t]_{xyk}$ is calculated as

$$\begin{aligned} \mathbf{G}^{(1)}[t] &= \sigma(\text{BN}(\mathbf{U}^{(1)} * \mathbf{H}^{(2)}[t-1])) \\ C^{(1)}[t]_{xyk} &= \text{BN}(\mathbf{W} * (\mathbf{G}^{(1)}[t] \odot \mathbf{H}^{(2)}[t-1]))_{xyk} \\ H^{(1)}_{xyk}[t] &= \zeta(X_{xyk} - C^{(1)}_{xyk}[t](\alpha_k H^{(2)}_{xyk}[t-1] + \mu_k)) \end{aligned} \quad (16)$$

where

$$\text{BN}(\mathbf{h}; \delta, \nu) = \nu + \delta \odot \frac{\mathbf{h} - \widehat{\mathbb{E}}[\mathbf{h}]}{\sqrt{\widehat{\text{Var}}[\mathbf{h}] + \epsilon}}$$

Where $\mathbf{h} \in \mathbb{R}^d$ is the vector of preactivations being normalized by batchnorm and $\delta, \nu \in \mathbb{R}^d$ are parameters that control the standard deviation and mean of the normalized activities, ϵ is a regularization hyperparameter, and $\odot$ is elementwise multiplication. As in the hGRU described in the main text, this updated $\mathbf{H}^{(1)}[t]$ is next used to calculate $\mathbf{H}^{(2)}[t]$.

$$\mathbf{G}^{(2)}[t] = \sigma(\text{BN}(\mathbf{U}^{(2)} * \mathbf{H}^{(1)}[t])) \quad (17)$$

$$C^{(2)}[t]_{xyk} = \text{BN}(\mathbf{W} * \mathbf{H}^{(1)}[t])_{xyk} \quad (18)$$

$$\tilde{H}_{xyk}^{(2)}[t] = \zeta(\kappa_k H_{xyk}^{(1)}[t] + \beta_k C_{xyk}^{(2)}[t] + \omega_k H_{xyk}^{(1)}[t] C_{xyk}^{(2)}[t]) \quad (19)$$

$$H_{xyk}^{(2)}[t] = H_{xyk}^{(2)}[t-1](1 - G_{xyk}^{(2)}[t]) + \tilde{H}_{xyk}^{(2)}[t] G_{xyk}^{(2)}[t] \quad (20)$$

Performance of the hGRU-BN can be found in S4 as “hGRU (batchnorm)”. In addition to its ability to solve the Pathfinder challenge, the hGRU-BN is far more stable during training than the original hGRU formulation when its pointwise nonlinearity ζ is substituted for nonlinearities that are not squashing. This allowed us to build a version of the hGRU where the pointwise nonlinearity ζ is set to linear rectification (ReLU), which constrains $\mathbf{H}^{(1)}$ to inhibition and $\mathbf{H}^{(2)}$ to excitation as is done in the original model by [20]. Also similar to [20] this model used separate kernels $\mathbf{W}^I$ and $\mathbf{W}^E$ to calculate facilitative vs. suppressive horizontal interactions at $\mathbf{H}^{(1)}$ and $\mathbf{H}^{(2)}$. This model solved the pathfinder challenge, and is denoted in Fig. S4 as “hGRU (nonnegative)”.

We also tested whether the hGRU’s ability to solve the Pathfinder challenge was merely a function of its two processing stages. We developed a GRU with two layers of convolution (with separate kernels) to calculate its candidate hidden state. Although this “GRU (2L)” model performed better than the typical one-layer version on the 6-length pathfinder, it was unable to solve the other versions of the problem (Fig. S4).

Additional models We extended our screening of feedforward architectures on the Pathfinder challenge to include three new model classes. The first combines highway network modules [71] with the input- and readout-stages that were used to screen feedforward model operations in the main text. As with those models, we tested the highway network in one-, three-, and five-layer configurations. The highway network models performed similarly to the “large-kernel” configuration of feedforward models screened in the main text, solving the 6- and 9-length pathfinder tasks, but straining on the 14-length one (see “Hwy-Net” in Fig. S4).

We also tested “constrained” versions of our top-performing feedforward and per-pixel prediction models, the ResNet-152 and the U-Net. We created new versions of these models that had $\frac{1}{2}$, $\frac{1}{5}$ th, or $\frac{1}{10}$ th the total number of parameters. The resulting ResNet models solved the 6- and 9-length pathfinder datasets were unsuccessful on the 14-length pathfinder. In contrast, the constrained U-Nets performed far better, with only the U-Net $\frac{1}{10}$ strained by the 14-length pathfinder. Plotting model performance as a function of their multiple of parameters vs. the hGRU reveals a performance shelf for these feedforward models that depend on depth to expand their receptive fields and solve tasks such as the Pathfinder challenge. This strategy is less efficient than the hGRU’s ability to form interactions between units *at a single processing layer*. The model with the closest performance and number of parameters as the hGRU in this comparison is the $\frac{1}{5}$ U-Net, which still needed over an order of magnitude more free parameters to solve the challenge.

Training on natural images and comparisons to human data

Visualizing horizontal connections Theoretical models of visual cortex suggest that patterns of horizontal connections reflect statistical regularities of oriented features in natural scenes [55]. Here too we find that horizontal kernels in the hGRU learned from both synthetic images from the Pathfinder challenge and natural scenes resemble cortical patterns of horizontal connectivity, including association fields and the paired near-excitatory and far-inhibitory surrounds (Fig. 5b).

We visualized hGRU horizontal connectivity in models through a two-step procedure that aligned kernels into a common reference orientation (90 degrees), and then used PCA to identify and denoise common patterns. For each input channel of the hGRU’s horizontal kernel, we rotated the kernels

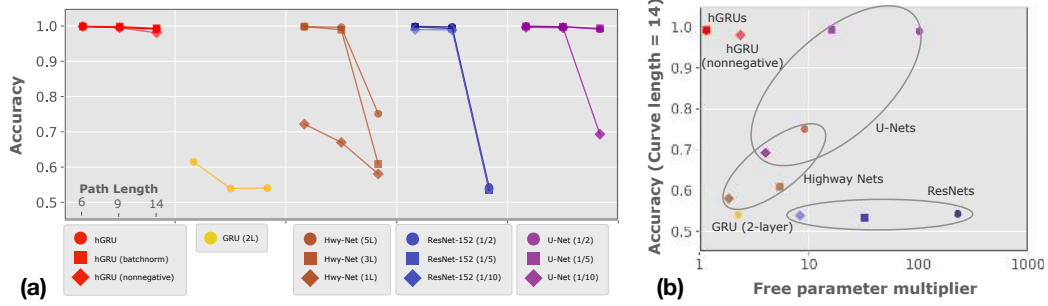


Figure S4: A comparison of the hGRU and additional control models on the Pathfinder challenge. These models are hGRUs with additional normalization stages (“batchnorm”) or constraints (“non-negative”); a GRU with two layers of processing; feedforward models with highway network layers; 152-layer ResNets restricted to a fraction of the total parameters of the original model; and U-Net models restricted to a fraction of the total parameters of the original model. (a) While all versions of the hGRU have high accuracy on all three Pathfinder challenge datasets, only the U-Net limited to $\frac{1}{2}$ or $\frac{1}{5}$ the total number parameters performs similarly. (b) Model accuracy is plotted as a function of the multiple of parameters each has w.r.t. the hGRU (i.e., hGRU is 1 and all models with more parameters are > 1). This reveals a model complexity and performance gap between the hGRU and other classes of feedforward models. Per-pixel prediction models, such as the U-Net, perform relatively well on the Pathfinder challenge but need many more parameters than the hGRU to do so: note the difference in performance between the U-Net ($\frac{1}{10}$) and the hGRU. Highway nets seem to follow a similar performance trajectory, and 152-layer ResNets struggle with the task without their full set of parameters (see Fig. 3 in the main text).

in the opposite orientation of the previous layer’s filter used to compute their feedforward drive. For example, for hGRUs trained on the Pathfinder challenge, the feedforward drive came from kernels initialized with oriented gabors. Here, a gabor oriented around 90 degrees would result in its horizontal kernels being rotated -90 degrees. After normalizing the orientation of all horizontal kernels, they were mean centered and then passed through PCA to visualize their “eigenconnectivity” patterns in Fig. 5. In other words, these eigenconnectivities correspond to the spatial associations learned by horizontal kernels, aggregated over all orientation channels. Eigenconnectivity patterns are sorted by their eigenvalues (80% cumulative variance cutoff).

We plot hGRU eigenconnectivities learned from two tasks: the Pathfinder challenge (Fig. 5a) and contour detection in natural images (Fig. 5b). For the first task, we use models described in the main text trained on each of the Pathfinder datasets. For the second task, we trained a new version of the hGRU to detect contours in the BSDS500 [1]. We used an 8-timestep with the first two blocks of PASCAL weight initialized convolutional layers from the VGG16 as its input processing stage. This model was trained for 1000 epochs on the 200 training images in the BSDS500, using data augmentations (random crops to 300×300 pixels, random rotations of $\pm 30^\circ$, and random up/down/left/right flips), a per-pixel cross-entropy loss, and the same learning rate and optimizer as the models described above.

Pathfinder challenge eigenconnectivity is similar to canonical patterns of cortex, including the (1) the antagonistic near-excitatory vs. far-inhibitory surround organization of hypercolumns in the visual cortex [56]; (2) the association field, with collinear excitation and orthogonal inhibition [8, 9]; and (3) higher-order surround computations [57].

Horizontal kernel eigenconnectivities are far cleaner and qualitatively more similar to purported canonical patterns of connectivity in cortex after training the hGRU to detect contours in natural images (compare Fig. 5a and Fig. 5b). We believe that such a difference at least in part results from the the kinds of image statistics that are useful for the respective tasks of Pathfinder vs. BSDS – path integration vs. contour detection. The path integration task in the Pathfinder challenge can be solved by relying nearly exclusively on the co-linearity of neighboring paddles whereas contour detection in natural images might oftentimes require richer, sometimes non-directional, measures of local change.

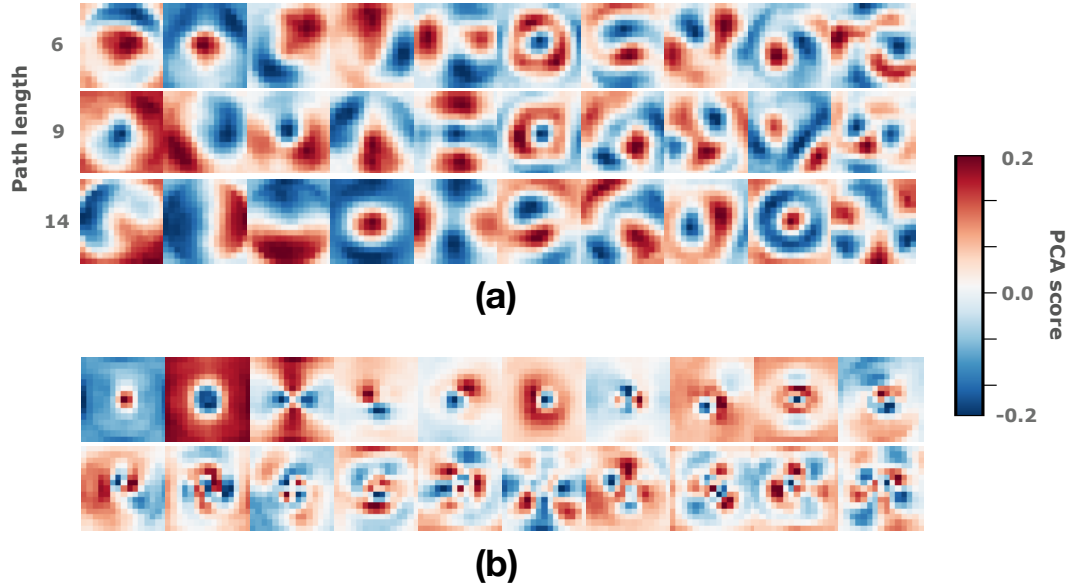


Figure S5: hGRU eigenconnectivity on when trained on the Pathfinder challenge and contour detection in natural scenes. (a) Eigenconnectivity of hGRU models trained on 6-, 9-, and 14-length Pathfinder datasets. (b) Eigenconnectivity of an hGRU on the BSDS500 contour detection dataset. The exhibited kernels contain patterns that are similar to those observed in visual cortex.

hGRU explains human contour detection The patterns of horizontal connectivity such as association fields are thought to underlie human performance in contour detection [64]. Given the presence of visually similar horizontal connectivity in the hGRU, we were motivated to test its ability to explain human psychophysics data during contour detection. We did this by recreating synthetic datasets from [22] that were used to measure human contour detection performance. The task involved detecting a contour of co-linearly aligned paddles in an array of randomly oriented distractor paddles. Different versions of the task were generated by varying inter-paddle distance within a contour (5 conditions).

We generated 1,000,000 unique 150×150 pixel images for every condition (28 total). In each case, 90% of images (900,000) were used for training and the remaining 10% (100,000) for testing. We fine-tuned our BSDS-trained hGRU separately on each of these training datasets and recorded its accuracy on the corresponding test datasets for comparison with human observers.

Although the hGRU was accurate at detecting paddles when the distance between paddles increased, this manipulation still strained network performance. We compared hGRU performance with human participants, whose responses were digitally extracted from [22] and averaged together for every experimental condition. Plotting the accuracy of the hGRU against the reported “detection score” of human observers revealed similar straining for both in response to increasing inter-paddle distance (Fig. S6).

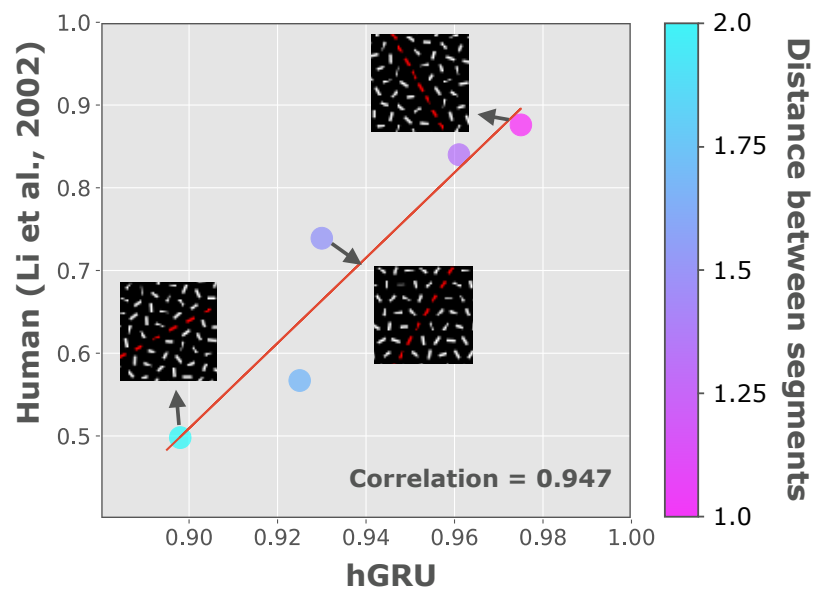


Figure S6: An hGRU trained on contour detection in natural images and then fine-tuned on a classic contour detection task performs similarly to human observers. While the hGRU is overall more accurate than humans, the performance of both is strained as the distance between paddles increases. Inlays show example contour stimuli with varying distance between their paddles (from left to right: larger-to-smaller gaps). Note that we use red to highlight contour stimuli for visualization, but it is absent during the task.